BASE DE DATOS 23 de Abril del 2026

Pim-Pam Boing

Simulación de datos

Samantha Salazar Fernández
Liliana Rosales Gómez
Angelica Citlali Gutiérrez Ortega
Adriana Rosales Gayosso
Ilse Guadalupe Saavedra Villa

Simulación de Datos

1.Procedimientos Almacenados

Los procedimientos definidos actúan como la "capa lógica" que permite al videojuego interactuar con la base de datos de forma segura y estandarizada.

Procedimiento: sp_simulacion_pinpon_v2

- **Mecanismo:** Es un Procedimiento Almacenado complejo que utiliza estructuras de control de flujo (WHILE, IF, CASE), variables locales y funciones de aleatoriedad.
- **Propósito:** Automatiza la creación de un ecosistema completo de datos. Simula el registro de personas (físicas y morales), la creación de sus usuarios, la configuración de métodos de pago, el procesamiento de transacciones financieras, la asignación de licencias y la generación de actividad de juego (perfiles de niños y sus sesiones).
- **Garantía de Coherencia y Generación de Datos:**
    1. **Lógica de Negocio Integrada:** No genera datos al azar sin sentido; por ejemplo, solo crea perfiles de niños y sesiones de juego si la transacción financiera fue 'Aprobada'.
    2. **Coherencia Temporal:** Calcula fechas lógicas (ej. el método de pago se registra minutos después de la persona, y la sesión de juego ocurre después de que la licencia está activa).
    3. **Diferenciación de Entidades:** Implementa lógica condicional para distinguir entre una 'Persona Física' y una 'Persona Moral/Institucional', asegurando que el volumen de datos generado respete la naturaleza del cliente.
    4. **Validación de Estatus:** Garantiza la integridad de los estados; por ejemplo, si una tarjeta está 'Expirada', el procedimiento fuerza el rechazo de la transacción y la cancelación de la licencia.

2.Funciones

Se implementaron funciones para realizar cálculos dinámicos que el sistema necesita consultar frecuentemente:

F1: fn_licencia_vigente

- **Propósito:** Determina si un usuario tiene permiso activo para jugar en el momento exacto de la consulta.
- **Justificación:** Garantiza la coherencia de acceso. Evita que se generen sesiones de juego para usuarios con licencias vencidas o canceladas, actuando como un filtro de integridad entre el módulo financiero y el módulo de juego.

F2: fn_dias_restantes_licencia

- **Propósito:** Calcula el tiempo de vida útil que le queda a la suscripción actual.

- **Justificación:** Proporciona coherencia temporal. Permite al sistema generar alertas proactivas antes de que los datos de acceso caduquen, asegurando que la continuidad del servicio esté respaldada por registros de fechas válidos.

F3: fn_total_sesiones_nino

- **Propósito:** Cuantifica el éxito de un perfil contando únicamente las sesiones con estatus 'Completado'.
- **Justificación:** Genera datos de rendimiento. Al filtrar por estatus, asegura que la estadística de progreso sea veraz y no se vea alterada por intentos fallidos o abandonos, manteniendo la integridad del historial del jugador.

F4: fn_promedio_fallos_nino

- **Propósito:** Calcula la media aritmética de errores cometidos por un niño en todas sus sesiones.
- **Justificación:** Es un mecanismo de monitoreo de calidad. Permite detectar si los datos generados por las sesiones indican una dificultad excesiva, garantizando que el análisis del comportamiento del usuario sea estadísticamente coherente.

F5: fn_nivel_mas_jugado

- **Propósito:** Identifica la preferencia del usuario mediante la frecuencia de registros por nivel.
- **Justificación:** Crea perfiles de usuario coherentes. Transforma una lista de miles de registros de sesiones en una preferencia clara (un solo dato), lo que facilita la personalización de la experiencia basada en evidencia histórica real.

F6: fn_tasa_abandono_nino

- **Propósito:** Obtiene el porcentaje de sesiones que el usuario no terminó.
- **Justificación:** Actúa como un indicador de retención. Garantiza que el sistema pueda identificar inconsistencias entre el uso del juego y la satisfacción del usuario, detectando patrones de desmotivación a través de la relación entre sesiones totales y abandonadas.

F7: fn_ingresos_persona

- **Propósito:** Suma el monto total de los pagos exitosos realizados por un cliente.
- **Justificación:** Garantiza la coherencia financiera. Al ignorar transacciones 'Rechazadas' o 'Pendientes', asegura que los datos de ingresos generados para reportes contables coincidan exactamente con el flujo de dinero real aprobado.

3.Transacciones

Análisis de Transacciones

- **Mecanismo:** El uso de bloques transaccionales (START TRANSACTION, COMMIT, ROLLBACK) y el control de claves foráneas (FOREIGN_KEY_CHECKS).
- **Propósito General:** Garantizar que si una operación falla, ninguna de las operaciones relacionadas se guarde, evitando que la base de datos quede en un estado inconsistente (por ejemplo, un niño jugando sin que se haya registrado su pago).

1. Atomicidad en el Registro de Suscripción

- **Descripción:** Cuando se genera una nueva licencia, el sistema vincula la inserción en transacciones_financieras con la de licencias.
- **Justificación:** Garantiza la coherencia financiera. Asegura que no exista una licencia "Vigente" si no hay una transacción "Aprobada" que la respalde. Si el registro de la transacción falla, la licencia no se genera.

2. Coherencia en la Generación de Actividad

- **Descripción:** El procedimiento utiliza la lógica de transacción para condicionar la creación de perfiles y sesiones.
- **Justificación:** Este mecanismo asegura la **integridad lógica**. Evita la generación de sesiones de juego para usuarios que no completaron su proceso de pago, manteniendo la base de datos limpia y veraz.

3. Control de Integridad Referencial Temporal

- **Descripción:** El uso de SET FOREIGN_KEY_CHECKS = 0 y 1 durante la generación masiva de datos.
- **Justificación:** Aunque es un mecanismo técnico, permite la generación eficiente de datos complejos en bloque, garantizando que al finalizar el proceso (SET = 1), todas las relaciones entre padres e hijos (Persona -> Usuario -> Transacción -> Licencia) sean validadas por el motor de la base de datos.

4. Sincronización de Estatus

- **Descripción:** La transacción garantiza que el estatus del método de pago se refleje inmediatamente en el estatus de la transacción y, por ende, en la validez de la licencia.
- **Justificación:** Mantiene la coherencia de estados. Si una tarjeta es rechazada en la transacción, el mecanismo asegura que el flujo de datos se detenga y no se generen registros de progreso posteriores para ese perfil.

4.Triggers

Los triggers se configuraron para aplicar reglas de seguridad que no pueden ser saltadas manualmente:

Tabla habilidades

1. Trigger: tr_hab_ins

- **Operación:** AFTER INSERT
- **Descripción:** Cada vez que se añade una nueva habilidad (como 'Memoria' o 'Lógica'), el trigger inserta un registro en la tabla bitacora.
- **Justificación:** Garantiza la coherencia de origen. Permite saber exactamente qué datos nuevos se están generando en el sistema y asegura que la tabla de bitácora sea un reflejo fiel de las inserciones en el catálogo de habilidades.

2. Trigger: tr_hab_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Captura el estado previo del dato usando el prefijo OLD.nombre y registra que hubo una edición.
- **Justificación:** Garantiza la coherencia de cambio. En un entorno de examen, esto demuestra que el sistema protege la integridad histórica, permitiendo rastrear qué habilidades fueron modificadas para evitar cambios no autorizados o accidentales sin supervisión.

3. Trigger: tr_hab_del

- **Operación:** AFTER DELETE
- **Descripción:** Antes de que el dato desaparezca por completo, el trigger guarda el nombre de la habilidad borrada en la bitácora.
- **Justificación:** Garantiza la trazabilidad de pérdida de datos. Asegura que la eliminación de un registro no sea "invisible". Para la coherencia de datos, es vital saber qué información dejó de existir para poder realizar auditorías forenses si algo falla en la lógica del negocio.

Tabla licencias

1. Trigger: tr_lic_ins

- **Operación:** AFTER INSERT
- **Descripción:** Registra en la bitácora la creación de una nueva licencia, vinculándola inmediatamente con el ID de la persona.

- **Justificación:** Garantiza la coherencia de asignación. Al dejar constancia de qué persona recibió qué licencia, asegura que el procedente del pago sea rastreable, evitando la existencia de accesos "fantasma" sin registro de origen.

2. Trigger: tr_licencias_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Monitorea cambios críticos, especialmente en el campo estatus (ej. de 'Vigente' a 'Vencida' o 'Cancelada').
- **Justificación:** Garantiza la coherencia de estatus. Es vital para la rúbrica porque asegura que la base de datos mantenga la integridad de las reglas de negocio; si una licencia se cancela manualmente, el trigger guarda quién hizo el cambio, protegiendo la lógica de acceso al juego.

3. Trigger: tr_licencias_delete

- **Operación:** AFTER DELETE
- **Descripción:** Registra la eliminación física de una licencia de la base de datos.
- **Justificación:** Garantiza la integridad histórica. Dado que las licencias representan ingresos y permisos de acceso, su borrado es una acción crítica. Este mecanismo asegura que incluso si un dato desaparece de la tabla principal, la bitácora mantenga la coherencia de que ese registro existió y fue eliminado por el Admin_Sistema.

Tabla métodos pago

. Trigger: tr_met_ins

- **Operación:** AFTER INSERT
- **Descripción:** Almacena la red bancaria y el dueño del método recién creado.
- **Justificación:** Garantiza la coherencia de procedencia financiera. Permite verificar que el flujo de datos que viene desde la interfaz de pagos se registró correctamente, vinculando la entidad bancaria con el ID de la persona para auditorías de fraude.

2. Trigger: tr_met_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Registra cualquier cambio en los datos de la tarjeta o cuenta del usuario.
- **Justificación:** Garantiza la coherencia de integridad. Es fundamental para tu examen explicar que este trigger evita que los cambios en métodos de pago pasen desapercibidos, asegurando que si un usuario o administrador altera una cuenta, exista un registro de quién (Pagos) y qué registro se afectó.

3. Trigger: tr_metodos_pago_del

- **Operación:** AFTER DELETE
- **Descripción:** Documenta la baja de una tarjeta o cuenta del sistema.
- **Justificación:** Garantiza la trazabilidad de eliminación. En el manejo de datos financieros, el borrado de registros es crítico; este mecanismo asegura que la base de datos mantenga la coherencia histórica, permitiendo saber que un método de pago existió aunque ya no esté disponible para nuevas transacciones.

Tabla niveles

1. Trigger: tr_niv_ins

- **Operación:** AFTER INSERT
- **Descripción:** Registra el nombre del nuevo nivel o microjuego que se añade a la plataforma.
- **Justificación:** Garantiza la coherencia en la expansión de datos. Permite verificar que los nuevos contenidos generados por el equipo de desarrollo se integren correctamente en el catálogo, manteniendo un historial limpio de cuándo se habilitó cada nivel.

2. Trigger: tr_niveles_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Monitorea cambios en el orden de los niveles o en su estatus (si son Premium o gratuitos), atribuyendo el cambio al usuario Game_Designer.
- **Justificación:** Garantiza la coherencia en la experiencia de usuario. Al auditar cambios en el orden o tipo de nivel, el sistema asegura que la progresión del niño no se vea alterada de forma arbitraria sin dejar un rastro que permita corregir posibles errores de lógica en el juego.

3. Trigger: tr_niv_del

- **Operación:** AFTER DELETE
- **Descripción:** Documenta la salida de un nivel del catálogo activo.
- **Justificación:** Garantiza la integridad referencial histórica. Dado que muchos niños pueden tener registros de progreso en un nivel específico, su eliminación es una acción crítica; este mecanismo asegura que se sepa qué contenido fue retirado para justificar posibles vacíos en las estadísticas de los perfiles.

Tabla perfiles jugadores

1. Trigger: tr_pro_ins

- **Operación:** AFTER INSERT
- **Descripción:** Al momento de registrar un nuevo perfil de niño, guarda en la bitácora el apodo elegido para el jugador.
- **Justificación:** Garantiza la coherencia en el registro de usuarios. Permite validar que el flujo de creación de perfiles sea constante y rastreable, asegurando que cada nuevo jugador generado tenga un respaldo en los registros de auditoría del sistema.

2. Trigger: tr_pro_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Detecta y documenta cualquier cambio realizado en la información del perfil de un niño (como cambio de apodo o avatar).
- **Justificación:** Garantiza la coherencia de integridad de perfil. Asegura que las modificaciones en la identidad digital del usuario no sean anónimas, permitiendo al administrador verificar la evolución de los datos generados por el usuario final.

3. Trigger: tr_pro_del

- **Operación:** AFTER DELETE
- **Descripción:** Registra la eliminación definitiva de un perfil de jugador de la base de datos.
- **Justificación:** Garantiza la trazabilidad de la actividad. Debido a que los perfiles son la base para generar estadísticas de juego y progreso, su eliminación debe estar estrictamente auditada para justificar la desaparición de los datos de desempeño asociados a dicho niño.

Tabla persona física

1. Trigger: tr_pf_ins

- **Operación:** AFTER INSERT
- **Descripción:** Registra en la bitácora el nombre de la persona física en el momento en que sus datos son creados en el sistema.
- **Justificación:** Garantiza la coherencia en la generación de identidades. Permite auditar el flujo de registro inicial, asegurando que cada identidad legal generada tenga una correspondencia exacta y fechada en la bitácora de control.

2. Trigger: tr_pf_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Documenta cualquier actualización en los datos legales (como cambios de apellidos o correcciones de fecha de nacimiento) asociándolos al Admin.
- **Justificación:** Garantiza la coherencia de veracidad de datos. Debido a que estos datos son la base de los contratos y facturación, este mecanismo asegura que cualquier cambio en la "verdad" legal del usuario esté debidamente registrado para evitar suplantaciones o errores no rastreados.

3. Trigger: tr_pf_del

- **Operación:** AFTER DELETE
- **Descripción:** Registra la eliminación del registro de identidad física, identificando el ID de la persona afectada.
- **Justificación:** Garantiza la trazabilidad de integridad referencial. Al ser una tabla que complementa a la tabla personas, su borrado debe ser monitoreado para entender por qué una entidad general perdió sus atributos específicos, manteniendo la coherencia histórica del sistema.

Tabla persona moral

1. Trigger: tr_pm_ins

- **Operación:** AFTER INSERT
- **Descripción:** Registra en la bitácora la razón social de la nueva institución o empresa que se ha dado de alta en el sistema.
- **Justificación:** Garantiza la coherencia en la captación de clientes. Al registrar automáticamente la creación de entidades morales, el sistema asegura que el crecimiento de la base de datos institucional sea auditable y coincida con los registros de contratos generados.

2. Trigger: tr_persona_moral_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Documenta cambios críticos en la información fiscal, como la razón social o el RFC, atribuyendo la responsabilidad al usuario Admin_Soporte.
- **Justificación:** Garantiza la coherencia de validez legal. Debido a que las personas morales tienen obligaciones fiscales específicas, este mecanismo asegura que cualquier alteración en sus datos de identificación quede plasmada en la bitácora, previniendo errores en la facturación o reportes tributarios.

3. Trigger: tr_pm_del

- **Operación:** AFTER DELETE

- **Descripción:** Registra la eliminación de una entidad moral, guardando su razón social antes de que el registro desaparezca de la tabla principal.
- **Justificación:** Garantiza la trazabilidad de la estructura organizacional. Dado que una persona moral puede tener vinculados a muchos niños, su eliminación es un evento de alto impacto; este disparador asegura que exista evidencia histórica de la baja de la institución para futuras auditorías o aclaraciones de servicio.

Tabla personas

1. Trigger: tr_per_ins

- **Operación:** AFTER INSERT
- **Descripción:** Al crear una nueva entidad, registra su identificador único y si se trata de una persona física o moral.
- **Justificación:** Garantiza la coherencia en el flujo de registro. Al capturar el "Tipo" de persona desde el inicio, el sistema asegura que la generación de datos sea consistente con las tablas hijas, permitiendo auditar si el proceso de registro siguió el camino lógico correcto.

2. Trigger: tr_per_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Registra cambios en el estado de la persona (ej. de 'Activo' a 'Suspendido'), vinculando la acción al usuario Admin.
- **Justificación:** Garantiza la coherencia de operatividad. Este mecanismo es vital para la seguridad, ya que permite rastrear por qué se le restringió el acceso a una entidad base, asegurando que los cambios de estatus en los datos generados tengan un respaldo histórico claro.

3. Trigger: tr_personas_delete

- **Operación:** AFTER DELETE
- **Descripción:** Documenta la eliminación definitiva de una entidad raíz del sistema, bajo la responsabilidad de un Admin_Supremo.
- **Justificación:** Garantiza la trazabilidad de integridad estructural. Debido a que borrar una fila en personas puede activar borrados en cascada en muchas otras tablas (usuarios, licencias, perfiles), este disparador actúa como el último registro de seguridad para justificar la eliminación de toda una rama de datos asociados.

Tabla planes licencia

1. Trigger: tr_plan_ins

- **Operación:** AFTER INSERT
- **Descripción:** Registra en la bitácora el nombre del nuevo esquema de suscripción (ej. 'Mensual', 'Anual') creado por el departamento de ventas.
- **Justificación:** Garantiza la coherencia en la oferta de servicios. Asegura que el catálogo de productos disponibles para los usuarios finales tenga un origen trazable, permitiendo auditar cuándo y qué tipo de planes se integraron al sistema de cobro.

2. Trigger: tr_planes_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Monitorea específicamente los cambios en el costo de los planes, registrando el precio anterior y el nuevo bajo la autoría de Admin_Ventas.
- **Justificación:** Garantiza la coherencia de integridad financiera. Es un control crítico para evitar cambios de precios no autorizados o erróneos; este mecanismo asegura que si el monto de una licencia cambia, exista una explicación histórica que respalde la diferencia en los ingresos generados.

3. Trigger: tr_plan_del

- **Operación:** AFTER DELETE
- **Descripción:** Documenta la eliminación de un plan de suscripción del catálogo activo.
- **Justificación:** Garantiza la trazabilidad del modelo de negocio. Debido a que las licencias históricas dependen de estos planes, registrar su eliminación es vital para entender por qué ciertos productos ya no están disponibles para nuevas ventas, manteniendo la coherencia de los reportes administrativos.

Tabla progreso niveles

1. Trigger: tr_pno_ins

- **Operación:** AFTER INSERT
- **Descripción:** Registra en la bitácora el inicio de un nuevo avance para un perfil de jugador específico.
- **Justificación:** Garantiza la **coherencia en la generación de historial**. Al documentar el momento en que un niño comienza a interactuar con un nivel, el sistema asegura que el flujo de datos de aprendizaje tenga un punto de partida auditable y vinculado correctamente al ID del perfil.

2. Trigger: tr_pno_upd

- **Operación:** AFTER UPDATE

- **Descripción:** Captura los cambios de estado en el progreso, como cuando un nivel pasa de 'Desbloqueado' a 'Completado'.
- **Justificación:** Garantiza la **coherencia de logro**. Este mecanismo es fundamental para validar que los datos generados por el motor de juego (el éxito del niño) se reflejen de forma fidedigna en la base de datos, permitiendo rastrear la transición de los estados de aprendizaje bajo el usuario Game.

3. Trigger: tr_progreso_delete

- **Operación:** AFTER DELETE
- **Descripción:** Documenta el borrado de registros de progreso, interpretándolo como un "Reset" de avance, bajo la supervisión de Admin_Soporte.
- **Justificación:** Garantiza la **trazabilidad de la experiencia del usuario**. Dado que el progreso es el valor principal para el usuario final, su eliminación debe estar justificada (por ejemplo, por una solicitud de soporte para reiniciar un curso), manteniendo la coherencia sobre por qué desaparecieron datos de avance histórico.

Tabla sesiones juegos

1. Trigger: tr_ses_ins

- **Operación:** AFTER INSERT
- **Descripción:** Almacena en la bitácora el aviso de inicio de sesión, vinculándolo al ID del perfil del niño que comenzó a jugar.
- **Justificación:** Garantiza la coherencia de uso del sistema. Permite validar que la generación de datos de actividad sea real y rastreable, asegurando que el administrador pueda ver en tiempo real qué perfiles están activos y cuándo iniciaron su sesión.

2. Trigger: tr_sesiones_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Registra el momento en que una sesión cambia de estado (normalmente de 'En curso' a 'Completado' o 'Abandonado'), atribuyendo la acción al Game_Engine.
- **Justificación:** Garantiza la coherencia de finalización de procesos. Al documentar el cierre de las sesiones, el sistema asegura que los datos generados (como el tiempo empleado o los fallos) tengan un respaldo de auditoría que confirme que el proceso de juego concluyó según la lógica del motor del juego.

3. Trigger: tr_ses_del

- **Operación:** AFTER DELETE
- **Descripción:** Documenta la eliminación de una sesión de juego específica del historial activo.

- **Justificación:** Garantiza la trazabilidad de la integridad de datos. Debido a que las sesiones son la base para calcular KPIs (como la tasa de abandono o el promedio de fallos), su eliminación debe ser monitoreada para evitar que se manipulen artificialmente las estadísticas de rendimiento de los jugadores.

Tabla transacciones financieras

1. Trigger: tr_trans_ins

- **Operación:** AFTER INSERT
- **Descripción:** Registra en la bitácora el monto de cada nuevo movimiento financiero que ingresa a través de la Pasarela_Pago.
- **Justificación:** Garantiza la coherencia de entrada financiera. Permite cruzar los datos generados por el proveedor de pagos externo con los registros internos de la base de datos, asegurando que cada intención de compra quede documentada con su valor exacto desde el primer segundo.

2. Trigger: tr_tra_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Captura los cambios de estado en una transacción, especialmente cuando pasa de 'Pendiente' a 'Aprobado' o 'Rechazado'.
- **Justificación:** Garantiza la coherencia de efectividad. Es un mecanismo vital para la rúbrica, ya que asegura que la transición del dinero (de un intento a un pago real) sea transparente, permitiendo auditar por qué se concedió o denegó el acceso a una licencia basándose en el historial de cambios de estatus.

3. Trigger: tr_tra_del

- **Operación:** AFTER DELETE
- **Descripción:** Documenta la eliminación de un registro de transacción, guardando el ID del movimiento afectado.
- **Justificación:** Garantiza la trazabilidad de integridad contable. El borrado de transacciones es una operación altamente sensible; este disparador asegura que no se pueda eliminar rastro de movimientos financieros sin dejar una evidencia en la bitácora, protegiendo al sistema contra fraudes o manipulaciones de ingresos.

Tabla usuarios

1. Trigger: tr_usu_ins

- **Operación:** AFTER INSERT
- **Descripción:** Registra la creación de una nueva cuenta, utilizando el propio email del usuario como referencia en la bitácora.

- **Justificación:** Garantiza la coherencia de autenticación. Permite confirmar el momento exacto en que una persona física o moral genera sus credenciales de acceso, asegurando que el inicio de su actividad digital sea plenamente rastreable.

2. Trigger: tr_usu_upd

- **Operación:** AFTER UPDATE
- **Descripción:** Documenta cambios sensibles como la actualización de contraseñas o el estado de la cuenta (estatus), identificando qué cuenta realizó el cambio.
- **Justificación:** Garantiza la coherencia de integridad de seguridad. Es fundamental para detectar accesos no autorizados o cambios de contraseña sospechosos, proporcionando un historial que respalda la validez de las credenciales vigentes.

3. Trigger: tr_usuarios_delete

- **Operación:** AFTER DELETE
- **Descripción:** Registra la eliminación permanente de una cuenta de usuario, guardando el email asociado para el registro histórico del Sistema/Admin.
- **Justificación:** Garantiza la trazabilidad de baja de usuarios. Dado que la eliminación de un usuario rompe el vínculo de acceso al sistema, este mecanismo asegura que exista evidencia de que la cuenta fue retirada, manteniendo la coherencia sobre quién tiene (o tuvo) acceso a la plataforma.

4.Indices

Mecanismo: Estructuras de aceleración **B-Tree** (Índices).

Propósito General: Garantizar que el sistema responda con velocidad inmediata ante consultas masivas, evitando cuellos de botella y asegurando que la búsqueda de información sea coherente con las necesidades del negocio (velocidad en login, precisión en finanzas y rapidez en el juego).

1. idx_usuarios_email

- **Campo:** usuarios(email)
- **Propósito:** Acelerar el proceso de autenticación (Login).
- **Justificación:** Garantiza la **coherencia en el acceso**. Al ser el correo el identificador principal para entrar al sistema, este índice evita escaneos completos de la tabla, permitiendo que el motor de la base de datos encuentre las credenciales en milisegundos, asegurando una validación de identidad eficiente.

2. idx_persona_apellido

- **Campo:** persona_fisica(primer_apellido)
- **Propósito:** Facilitar la búsqueda de personas por apellidos.
- **Justificación:** Mejora la **eficiencia administrativa**. Es vital para el servicio al cliente y soporte técnico, permitiendo localizar rápidamente a los responsables de las cuentas entre miles de registros, asegurando que la gestión de datos personales sea ágil y coherente.

3. idx_perfil_apodo

- **Campo:** perfiles_jugadores(apodo)
- **Propósito:** Filtrar perfiles de niños por su nombre de usuario o apodo.
- **Justificación:** Garantiza la **coherencia en la identidad del jugador**. Permite que el sistema cargue instantáneamente el avatar y las estadísticas personalizadas del niño al seleccionar su perfil, evitando retrasos en la experiencia de usuario final.

4. idx_transacciones_estatus

- **Campo:** transacciones_financieras(estatus)
- **Propósito:** Agilizar reportes financieros basados en el estado del pago.
- **Justificación:** Garantiza la **coherencia financiera**. Permite obtener el total de ingresos (Estatus 'Aprobado') de forma inmediata sin procesar transacciones rechazadas o pendientes, asegurando que los cálculos contables sean rápidos y precisos.

5. idx_transacciones_fecha

- **Campo:** transacciones_financieras(fecha_registro)
- **Propósito:** Buscar movimientos financieros por rangos de tiempo.
- **Justificación:** Facilita la **auditoría temporal**. Es clave para generar cierres de caja diarios, mensuales o anuales, permitiendo que el sistema genere reportes de crecimiento económico sin degradar el rendimiento del servidor.

6. idx_sesiones_dispositivo

- **Campo:** sesiones_juego(tipo_dispositivo)
- **Propósito:** Filtrar la actividad según el hardware usado (PC, Tablet, etc.).
- **Justificación:** Apoya la **coherencia tecnológica**. Permite al equipo de desarrollo identificar en qué dispositivos juegan más los niños para priorizar optimizaciones, transformando los datos operativos en información estratégica de manera veloz.

7. idx_sesiones_estatus

- **Campo:** sesiones_juego(estatus)
- **Propósito:** Filtrar sesiones completadas o abandonadas.
- **Justificación:** Garantiza la **coherencia en métricas de uso**. Este índice es el que permite que funciones como fn_tasa_abandono_nino funcionen rápidamente, facilitando la detección de niveles con dificultad excesiva o errores de diseño.

8. idx_progreso_estatus

- **Campo:** progreso_niveles(estatus)
- **Propósito:** Visualizar el estado de avance en los niveles educativos.
- **Justificación:** Garantiza la **coherencia pedagógica**. Acelera la carga del mapa de niveles para el niño, permitiendo que el juego sepa instantáneamente qué contenidos debe mostrar como 'Bloqueados' o 'Completados' según su progreso previo.

9. idx_bitacora_tabla

- **Campo:** bitacora(tabla)
- **Propósito:** Acelerar las búsquedas de auditoría por tabla específica.
- **Justificación:** Garantiza la **trazabilidad dirigida**. Permite que los administradores del sistema filtren todos los movimientos de una tabla crítica (como transacciones) en segundos, facilitando labores de depuración y vigilancia de datos.

10. idx_bitacora_operacion_fecha (Índice Compuesto)

- **Campos:** bitacora(operacion, fecha_hora)
- **Propósito:** Búsqueda avanzada por tipo de acción (Insert/Update/Delete) y tiempo.
- **Justificación:** Representa el máximo nivel de **coherencia en auditoría**. Al combinar la operación y la fecha, permite realizar análisis de los movimientos, asegurando que la supervisión del sistema sea eficiente incluso con millones de registros de log.